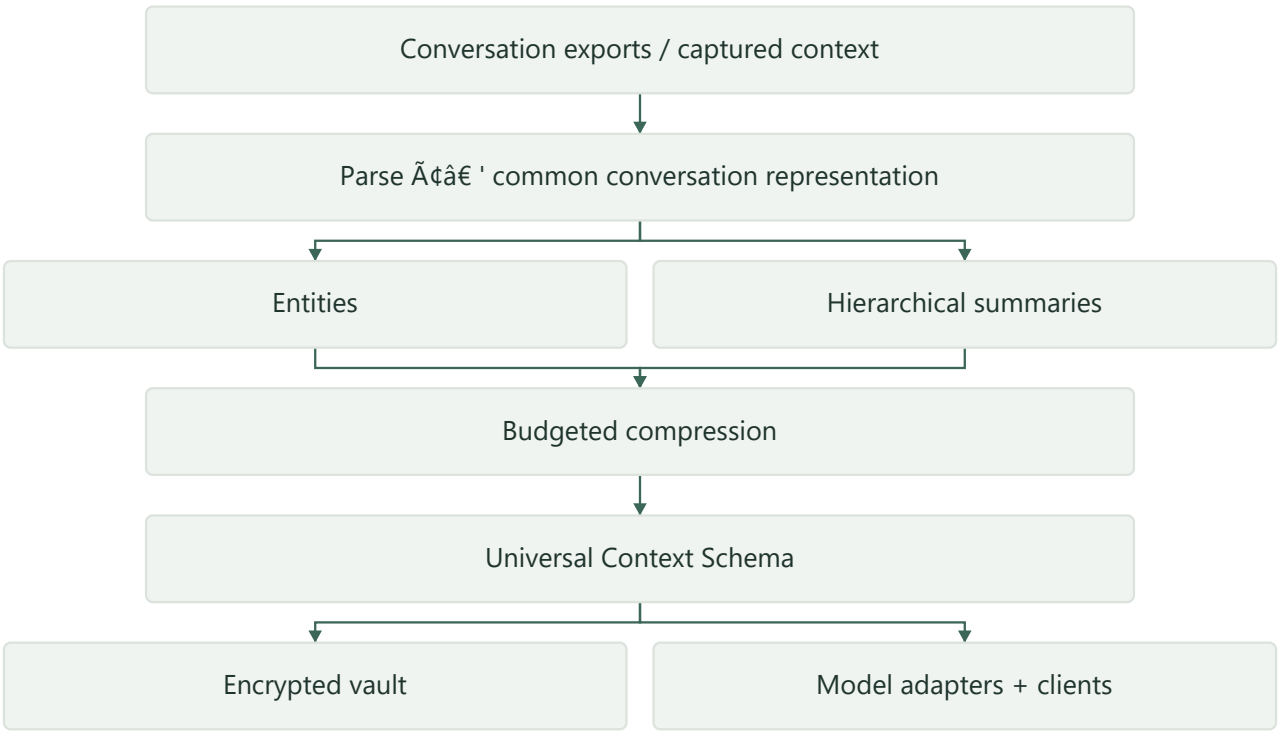
Kangaroo Shift

Context continuity across models and sessions

A context-processing platform that parses conversations, builds a structured context representation, compresses it, and supports storage and transfer.

Architecture



Component overview derived from the supplied source; version-specific details may differ.

Component	Implementation	Role
Processing engine	Parsers, entities, summaries	Structured conversation content
Compression	Priority and token-budget logic	Retain selected information
Schema	Universal Context Schema	Validation and transfer format
Storage	Vault + encryption	Versioned encrypted content
Application	FastAPI, Next.js, extension	User-facing workflows

Software prototype

Compression policy in detail

Kangaroo Shift converts a conversation into structured context before fitting it into a budget. The compressor uses a priority queue over selected summaries, entities and artifacts, while decisions and tasks are retained outside the droppable heap. The priorities express a continuity policy rather than learned estimates of future usefulness.

Source base priorities; higher is harder to remove. Entity priority = 2 * (0.5 + importance). The global summary is protected by category logic, not merely its numeric weight.

Budget arithmetic and ordering

For nonempty text, estimate_tokens returns max(1, floor(character_count/4)). It is an approximate budget, not the target model tokenizer. The heap removes lower priorities first; equal priorities compare indices so newer items are removed first. Thus the actual ordering can differ from the high-level docstring, especially for entities whose importance changes their priority.

Protected content and artifacts

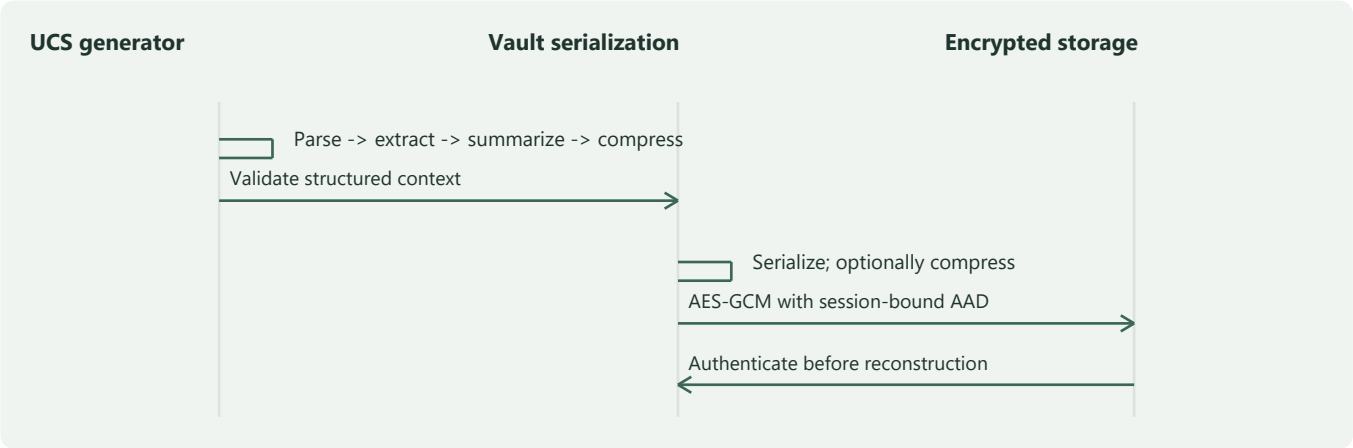
Decisions and tasks are never added to the heap. Global summaries are requeued as protected and can leave the result above budget when nothing droppable remains. Artifacts are first truncated toward a per-artifact cap; if later revisited without further token savings they can fall through to removal. That behavior is more nuanced than the comment saying artifacts are only truncated.

Design choice	Tradeoff
Protect tasks and decisions	Continuity is prioritized over a strict size guarantee
Character-based token estimate	Cheap and model-independent, but approximate
Prefix truncation	Preserves initial artifact text; later implementation details can be lost

Source / interpretation: kangaroo/backend/app/core/engine/compressor.py: estimate_tokens, PriorityItem.__lt__, truncate_artifact and CompressionPipeline.compress.

UCS assembly & encrypted storage

The UCS generator provides a structured interchange boundary between raw conversations and downstream clients. It coordinates parsing, entity extraction, hierarchical summaries, compression, schema assembly and validation. Storage is a separate concern: a structured context can be valid without its storage or transfer workflow being verified end to end.



Intermediate representation and diagnostics

The generation result preserves a common conversation representation and statistics including entity count, compression ratio, warnings and processing time. These are useful for tracing why a context shrank and what was retained. A schema-valid output still needs semantic testing: preserving a task field does not establish that its meaning or dependencies survived summarization.

Storage mechanism

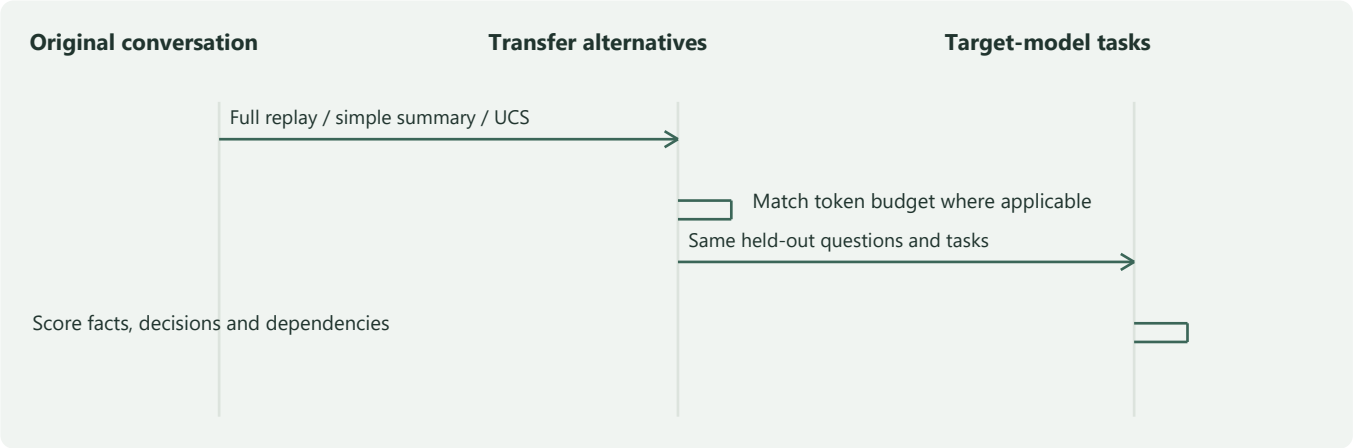
The inspected vault uses AES-GCM, PBKDF2-HMAC-SHA256-derived keys and session-bound associated authenticated data, with optional zlib compression. Associated data binds the encrypted content to a session context without becoming encrypted plaintext. The READMEâ€™s Argon2id label does not match this implementation. The vault and encryption module also use different iteration defaults; the effective runtime configuration must be identified before making a security claim.

Layer	Evidence it provides	Does not alone establish
Schema validation	Required structure and field types	Semantic completeness
Authenticated encryption	Integrity/authentication under correct key use	Production key management
Version metadata	Traceability of saved contexts	Successful cross-client migration

Source / interpretation: kangaroo/backend/app/core/engine/ucs_generator.py; storage/vault.py and encryption module. Diagram represents implementation responsibilities, not a security certification.

Transfer quality & application scope

The backend, web application and extension are substantial application surfaces around the context engine. Their existence supports a system-design account; an effectiveness claim needs a narrower experiment that isolates what a transferred context preserves at a given budget.



A proposed paired evaluation

Use the same conversation and held-out follow-up tasks across full replay, a simple summary baseline and Kangaroo-generated context. Record the target tokenizer, serialized size, factual recall, task completion and contradictory carryover. Plot performance against token budget rather than reporting only a compression ratio. This is an evaluation design for future evidence, not a completed comparison.

Implemented and scaffolded surfaces

The local source includes FastAPI routes, Next.js views, browser capture/synchronization and deployment configuration. The SDK client remains a module scaffold and the CLI health command prints a version string; neither should be presented as a fully working integration. The 251-file remote/local comparison confirms repository correspondence, not that every route or client has passed runtime tests.

Metric	Reason to include
Compression ratio	Shows size reduction, not preserved usefulness
Task/decision retention	Tests the stated continuity objective
Cross-client execution	Tests capture, persistence and restore as one workflow

Source / interpretation: Kangaroo engine, application, CLI and SDK source; remote/local file comparison. KS-Probe is related benchmark infrastructure, not substitute evidence for this paired transfer test.

Evidence & source

Implementation, supporting records, and the boundary of the claim

Area	Evidence	Reference
Repository check	251 remote files match local copies	245 exact; 6 line-ending normalized
Implementation	Backend, frontend, browser extension	Public repository source
Scaffolded surfaces	SDK and part of CLI incomplete	Local source inspection

Selected implementation files

File / record	Path / reference
ucs_generator.py	kangaroo/backend/app/core/engine/ucs_generator.py
compressor.py	kangaroo/backend/app/core/engine/compressor.py
vault.py	kangaroo/backend/app/core/storage/vault.py

The public repository is the full code reference. The selected snapshot highlights the context engine and vault.

Evidence boundary

The processing pipeline exists in source; context-preservation effectiveness requires controlled transfer experiments. The SDK client remains a scaffold in this snapshot.

Technical note

The compressor uses priority-based removal and approximate token counts. The vault implementation uses AES-GCM with PBKDF2-derived keys; this description follows source rather than the inconsistent README algorithm label.